M1: Approach

AccessCart AI / Accessible shopping across sight, hearing and motor access needs

STATUS Proposed milestone approach. The React web app and Bhashini integration are planned, not yet represented as implemented. Bhashini API registration/credentials are pending.

1 / Problem statement and understanding

Shopping interfaces often assume a person can see product images and controls, hear audio, and operate a screen precisely. Product specifications, reviews and checkout details can also arrive as dense text or visually structured content. For blind and low-vision shoppers, this can block product discovery and make price, delivery and purchase confirmation hard to verify. Hearing and hand-motor barriers create different breakdowns that a generic “accessibility mode” does not solve.

The brief calls for an accessible layer over a reference e-commerce shopping experience: users should browse categories, inspect product details, understand reviews and compare options, then complete a deliberate order flow. AccessCart AI will prioritize independent, informed choices, with confirmation before consequential actions.

2 / Idea and abstract

AccessCart AI is a planned, voice-first React web app for shopping in English, Telugu and Hindi. A shopper can speak a request, receive a concise and structured product answer in their chosen language, ask follow-up questions and proceed through accessible product comparison and checkout steps. Bhashini will provide the live multilingual speech path; the app will shape answers for the active access mode. The prototype will use a reference/catalog dataset and will not train an LLM from scratch.

Distinct, demonstrable adaptations

CONDITION	PLANNED ADAPTATION / DEMO
Visual impairment (mandatory)	Screen-reader-friendly semantic headings, labelled controls and keyboard flow; voice-first product browsing; speak price, delivery and review summaries; describe product images in words; read confirmation aloud before placing an order.
Hearing impairment	Every spoken exchange also appears as a persistent text transcript; show the assistant response as readable captions and use visual status/confirmation cues instead of audio-only feedback.
Hand-motor impairment	Voice commands for category, filter, compare and navigation actions; large, well-spaced controls and fewer steps; preserve typed/keyboard alternatives and avoid requiring precise gestures.

M1: Approach

Architecture, resilience and planned validation

3 / High-level approach and pipeline

A responsive browser client will run on a normal laptop or Android phone. Its accessible UI captures speech or text, maintains the selected language and access mode, presents product data with semantic structure, and keeps visible transcripts and status. A thin application service will coordinate catalog/search actions, answer formatting and Bhashini requests; secrets will remain server-side when integration is enabled.

1	2	3	4	5	6
Speech / text EN • TE • HI	**Bhashini ASR speech → text**	**Translate to working language**	**AI + catalog grounded answer**	**Translate to output language**	**Bhashini TTS text → speech**

Live speech-to-speech path required by the brief: ASR → translation → AI → translation → TTS. Input and output will support English, Telugu and Hindi, with same-language or cross-language responses. In hearing-access mode, transcript/captions remain available alongside speech.

Accessible answer design

Responses will use real headings and short lists, with an “explain simply” control. Product comparisons will be rendered as semantic text/table content and also summarized in words for speech. Image descriptions and review summaries will state useful details without relying on color or layout alone. Text size and speech rate will be adjustable. Text remains usable if audio is unavailable.

Reliability and graceful failure

For uncertain or misrecognized speech, show the recognized words and ask the shopper to confirm, edit or repeat before acting. Never place an order from an ambiguous command. On network/API timeout or drop, retain the transcript and current shopping context, announce that live speech is unavailable, and offer retry plus typed/keyboard navigation; clearly label any unavailable feature. Recover without silently repeating a purchase action.

4 / Planned testing methodology

Testing is planned for the prototype and will cover both task completion and accessibility behavior. Use scripted scenarios on laptop and Android-sized screens; test all three languages on both input and output; and run a short live Bhashini exchange once registration and credentials are available. Compare the raw catalog/assistant answer with its accessible rendering.

TEST AREA	PLANNED CHECK / EVIDENCE
Three conditions	Blind-first user completes browse → inspect → compare → confirm using keyboard/screen-reader and speech; hearing mode works from transcript/captions and visual cues without audio; motor mode completes key tasks with voice/large controls and minimal precise input.
Language + output	Exercise English, Telugu and Hindi input/output, including cross-language speech; inspect headings, lists, image descriptions, simpler explanations, comparison spoken summary, and adjustable text/speech speed.
Failure + safety	Inject low-confidence/misheard phrases and disconnect/timeout events; verify confirmation or repeat prompts, retained context, retry path, visible status and no unintended order or duplicate action.
Device + usability	Check responsive layout, keyboard focus/order, accessible labels, text scaling and readable contrast on a laptop and Android phone; record task completion, errors and time for iteration.

Scope and dependency Solo-built React web prototype; no special hardware and no LLM training. Bhashini credentials are not yet available, so live integration is a planned dependency—not a current capability. Validate the core UI and failure states with mocks until access is granted, then replace the mock path and demonstrate a live exchange.